

PulseAgent

Interview Defense & Project Ownership Document

Author	Sidharth Kriplani
Project URL	https://pulseagent-98058433335.us-central1.run.app
Corpus	6,221 WixQA help articles
Eval Set	200 queries · 56% ANSWER · 44% ABSTAIN · 75.9% citation precision
Stack	LangGraph · FastAPI · Groq (llama-3.3-70b) · fastembed · Qdrant · GCP Cloud Run
Purpose	Complete project ownership prep — architecture, decisions, failures, fundamentals, Q&A;

How to use this document: Read it cover-to-cover once. Then drill Section 5 (failures) and Section 8 (Q&A) until each answer is automatic. Section 3 (Key Concepts) is your fundamentals bridge — these are the concepts an interviewer will probe directly from your architecture. Section 7 (Truth Boundary) keeps you honest.

01 Project Summary

PulseAgent is a production-deployed agentic RAG system. It answers natural-language questions about Wix products by orchestrating four specialist agents — planner, retriever (parallel), NLI verifier, and synthesis — through a LangGraph StateGraph. It runs live on GCP Cloud Run and is callable via a REST API with API key auth.

What it does

- Accepts a natural-language question via POST /query (auth required)
- Decomposes the query into up to 3 sub-queries (LLM planner, JSON mode)
- Fans out parallel retrieval across sub-queries using LangGraph's Send() API
- Each retrieval run uses hybrid BM25 + dense (BAAI/bge-small-en-v1.5) + RRF fusion
- Aggregates, deduplicates, and re-ranks all retrieved chunks by RRF score
- Runs NLI citation verification (cross-encoder/nli-deberta-v3-small, threshold 0.85)
- Routes to ANSWER_WITH_CITATION if evidence passes, ABSTAIN if not
- Generates a grounded answer with a self-reflective critique loop (max 2 retries)
- Returns structured JSON: answer, citations, verdict, sub_queries, latency_ms, request_id

What it does NOT do

- It is not a general-purpose chatbot — scoped strictly to WixQA corpus
- It does not store conversations or maintain session state
- It does not do fine-tuning or retrieval augmentation beyond the static corpus
- The MCP server is implemented but not stress-tested in production
- The self-study (Supabase SR) layer from the original pulseknowledge repo is not wired in

Live URL	https://pulseagent-98058433335.us-central1.run.app
Auth	X-API-Key header. Key stored in GCP Secret Manager (pulse-api-key).
Rate limit	10 req/min per IP via slowapi
Logging	Structured JSON: request_id (uuid8), query (truncated 120), route, cited_count, latency_ms
Deployment	GCP Cloud Run · gcloud run deploy --source · scale-to-zero · revision pulseagent-00005-pds

LLM

Groq API · llama-3.3-70b-versatile · OpenAI-compatible endpoint ·
temperature=0.2

02 System Architecture

The system is a hierarchical multi-agent LangGraph StateGraph. One top-level supervisor graph orchestrates three compiled specialist subgraphs. All state flows through typed TypedDict schemas. The critical design point is the Annotated reducer on `all_retrieved_chunks`, which is what makes parallel fan-out accumulation work correctly.

Full Pipeline

```
POST /query
■
▼
planner_node (LLM, JSON mode)
■ Decomposes query → [sub_q1, sub_q2, sub_q3]
■ Returns Command(goto=[Send('retrieval_agent', {q}) for q in sub_queries])
■
■■■Send()■■■ retrieval_agent (sub_q1) ■■
■■■Send()■■■ retrieval_agent (sub_q2) ■■ parallel, independent
■■■Send()■■■ retrieval_agent (sub_q3) ■■
■
Annotated[List[dict], operator.add] ■■■ accumulates all chunks
■
▼
aggregate_node
■ Deduplicates by article_id (keep highest rrf_score)
■ Re-ranks, keeps top-K=10
▼
verifier_wrapper_node
■ Converts question → declarative claim
■ NLI check: cross-encoder/nli-deberta-v3-small
■ Threshold: confidence >= 0.85 AND verdict == SUPPORTS
■ Checks top 3 chunks only (TOP_K_VERIFY=3)
▼
synthesis_wrapper_node
■
■■■ generator_node → draft answer with citations
■■■ reflector_node → PASS or RETRY
■■■ (max 2 retries) → final_answer
■
▼
JSON response: {answer, citations, verdict, sub_queries, latency_ms}
```

State Schema

Every agent communicates through typed state. The supervisor state holds the complete lifecycle. Specialist agents have narrower states that write back into supervisor state via wrapper nodes.

```
class SupervisorState(TypedDict):
    query: str
    sub_queries: List[str]
    search_intent: str
    # Annotated reducer – parallel Send() runs ACCUMULATE, not overwrite
    all_retrieved_chunks: Annotated[List[dict], operator.add]
    verified_chunks: List[dict]
    contract_decision: str # ANSWER_WITH_CITATION | ABSTAIN | ERROR
    citation_precision: Optional[float]
    draft_answer: str
    final_answer: str
    cited_article_ids: List[str]
    reflection_notes: str
    reflection_passed: bool
    retry_count: int
    route: str
```

Node Responsibilities

Node	Responsibility	Key detail
planner_node	Query decomposition + fan-out	JSON mode LLM, json.loads() + fallback to original query
retrieval_agent (×N)	Hybrid BM25+dense RRF retrieval per sub-query	Compile subgraph, runs in parallel via Send()
aggregate_node	Dedup + re-rank	Keeps highest rrf_score per article_id, takes top 10
verifier_wrapper_node	NLI citation check	Cross-encoder on top 3 chunks, threshold 0.85
synthesis_wrapper_node	Answer generation + reflection	generator→reflector loop, max 2 retries

03 Key Concepts — Fundamentals Bridge

This section extracts the underlying concept behind each major component. An interviewer can pivot from 'walk me through your architecture' to any of these. Know each one well enough to explain it independently of the project.

3.1 BM25 — Why it handles exact technical terms

BM25 (Best Match 25, Okapi BM25) is a bag-of-words retrieval function based on TF-IDF. The key insight is its IDF term: words that appear in fewer documents get higher weight. For a product help corpus, exact identifiers like 'OAuth 2.0', 'SSL certificate', 'Premium plan' are rare across the corpus — BM25 scores them highly, exactly when a user query contains them. Dense embeddings encode semantic proximity but can miss exact rare tokens because the embedding space is continuous and approximate.

Formula	$\text{score}(q,d) = \sum \text{IDF}(q_i) \cdot [\text{tf}(q_i,d) \cdot (k_1+1)] / [\text{tf}(q_i,d) + k_1 \cdot (1 - b + b \cdot d /\text{avgdl})]$
Parameters	$k_1=1.5$ (term saturation), $b=0.75$ (length normalization) — BM25Okapi defaults
Strength	Exact keyword match, rare technical terms, product identifiers
Weakness	No semantic understanding — 'can I embed video' won't match 'add media to page'

3.2 Dense Retrieval — Bi-encoder vs Cross-encoder

Retrieval uses a bi-encoder. The query is embedded independently of the documents (BAAI/bge-small-en-v1.5 via fastembed ONNX). Dot-product or cosine similarity then ranks pre-computed document vectors. This is fast — the document vectors are computed once and stored.

The NLI verifier uses a cross-encoder (cross-encoder/nli-deberta-v3-small). A cross-encoder takes (query, document) as a single input and scores the pair jointly. It cannot be used for retrieval because you cannot pre-compute document vectors — each new query requires re-scoring every document. Cross-encoders are more accurate but $O(n)$ per query, so they're only applied to the top-K candidates retrieved by the bi-encoder. This is the standard re-ranking pipeline pattern.

Bi-encoder (retrieval)	Query and doc embedded separately. Fast. Used for BM25 + dense retrieval.
Cross-encoder (NLI)	Query+doc encoded jointly. Accurate. Applied to top 3 chunks only.
Why not cross-encoder for retrieval?	Cannot pre-compute doc vectors. $O(n)$ per query. Infeasible at 6,221 chunks.

3.3 Reciprocal Rank Fusion (RRF)

BM25 and cosine similarity scores are on incomparable scales — you cannot add them directly. Normalizing introduces a mixing hyperparameter. RRF sidesteps both problems by operating only on ranks.

```
rrf_score(d) =  $\sum_r 1 / (k + \text{rank}_r(d))$ 

k = 60 (smoothing constant from Cormack et al. 2009)
Sum over r = {dense_rank, bm25_rank}
If document not in a ranking, penalized with rank = (fetch_k + 1)
```

k=60 is the standard value from the original RRF paper. A document ranked 1st in both lists gets $\text{RRF} \approx 2/61 = 0.0328$. A document ranked 1st in one list and 60th in the other gets $\approx 1/61 + 1/120 = 0.025$. Documents strong in both modalities win, which is the desired behavior — exact-match AND semantic-match candidates are rewarded.

3.4 NLI Citation Verification

Natural Language Inference (NLI) is a classification task: given a premise and hypothesis, predict ENTAILMENT, CONTRADICTION, or NEUTRAL. The convention is premise = evidence (chunk), hypothesis = claim to be verified.

Critical detail: a user question is not a claim — it is interrogative, and NLI models are trained on declarative claims. Asking 'Can I add a payment method?' against a chunk results in poor entailment scores. The fix is claim conversion: strip the '?' and prepend 'This article provides information about: ...'. This converts the interrogative into a declarative hypothesis that NLI can evaluate.

```
# In verifier_agent.py:
raw = state['query'].strip().rstrip('?')
claim = f'This article provides information about: {raw}'

# NLI convention (SNLI/NLI standard):
# premise = chunk_text (the retrieved evidence)
# hypothesis = claim (the statement to be verified)
# Verdict: SUPPORTS | CONTRADICTS | NOT_ENOUGH_INFO

# Threshold: confidence >= 0.85 AND verdict == SUPPORTS
# Decision: ANSWER_WITH_CITATION if any verified, else ABSTAIN
```

3.5 LangGraph Send() API — Parallel Fan-out

In a normal LangGraph graph, edges are static: node A always goes to node B. The Send() API allows a node to return a Command with dynamic routing — specifying multiple target nodes, each with its own input state. These run in parallel (LangGraph executes them concurrently in the same step).

```

# planner_node returns:
return Command(
  update={'sub_queries': sub_queries, 'all_retrieved_chunks': []},
  goto=[
    Send('retrieval_agent', {'query': q, 'all_retrieved_chunks': []})
    for q in sub_queries # 1, 2, or 3 agents launched
  ]
)

# All retrieval_agent instances write to all_retrieved_chunks.
# Without the Annotated reducer, each would OVERWRITE the previous result.
# With operator.add, they ACCUMULATE correctly.

# SupervisorState:
all_retrieved_chunks: Annotated[List[dict], operator.add]

```

The `Annotated[List[dict], operator.add]` type annotation is not just a type hint — `LangGraph` reads it at runtime to decide how to merge concurrent writes to this state key. Without it, whichever `Send()` finishes last would overwrite the others. With it, all lists are concatenated. This is the key to making parallel fan-out work.

3.6 Thread-safe Singleton (Double-Checked Locking)

Cloud Run starts multiple request handler threads in the same process. The retrieval index (`_index`) and the compiled supervisor graph (`_supervisor`) are expensive to build — they must be built once and shared. The pattern is double-checked locking: check before the lock (fast path), acquire the lock, check again inside (in case another thread already built it).

```

_supervisor = None
_supervisor_lock = threading.Lock()

def run_supervisor(query: str) -> dict:
    global _supervisor
    if _supervisor is None: # fast path — no lock needed once set
        with _supervisor_lock:
            if _supervisor is None: # re-check: another thread may have finished
                _supervisor = build_supervisor()
    ...

```


04 Architectural Decision Log

Each decision below represents a real choice with real alternatives. The 'why' is the answer to give in an interview — not 'because it was easier' but because it fits the specific constraints of this system.

Decision 1: LangGraph over raw Python orchestration

Alternative considered: Hand-written loop with sequential LLM calls

Reasoning: LangGraph handles state typing, parallel fan-out via `Send()`, and compiled graph execution. A raw loop would require manual threading to parallelize retrieval, manual state passing between agents, and no graph introspection. The `Send()` API specifically is not easy to replicate cleanly — it lets each sub-query dispatch to an independent agent instance without shared mutable state.

Decision 2: RRF over score-based linear fusion

Alternative considered: Weighted sum: $\alpha \cdot \text{bm25_norm} + (1-\alpha) \cdot \text{cosine}$

Reasoning: BM25 scores and cosine similarity are on incomparable scales. Normalizing them (e.g., min-max scaling) introduces a mixing hyperparameter α that would require tuning. RRF operates only on ranks, is hyperparameter-free ($k=60$ is the standard from Cormack et al.), and empirically performs well when retrieval systems are complementary — which BM25 and dense retrieval are.

Decision 3: NLI cross-encoder over keyword overlap for citation verification

Alternative considered: Jaccard-style keyword overlap between claim tokens and chunk tokens

Reasoning: Keyword overlap is a lexical proxy — it rewards shared surface tokens, not entailment. A chunk saying 'Wix does not support X' would score high overlap for a query about X, which is the opposite of what we want. NLI at the semantic level checks actual entailment direction. The measured difference: keyword overlap would mark contradicting chunks as 'SUPPORTED'. NLI catches this.

Decision 4: ABSTAIN path instead of fallback answer

Alternative considered: Always generate an answer, even if NLI verification fails

Reasoning: The alternative silently hallucinates. If NLI threshold is not met, the retrieved chunks don't entail the query — generating an answer from them is fabrication with false confidence. ABSTAIN is a principled policy: if the system can't verify, it says so. This is better for trust calibration. The synthesis agent still runs on ABSTAIN but adds a 'best-effort' caveat.

Decision 5: fastembed (ONNX) over PyTorch sentence-transformers

Alternative considered: sentence-transformers with PyTorch backend

Reasoning: fastembed runs the embedding model as an ONNX graph via `onnxruntime`. No PyTorch, no CUDA overhead on CPU-only Cloud Run. The tradeoff: fastembed downloads 5 ONNX model files

(~130MB total) at first initialization, which caused the 2-minute cold start issue. The Dockerfile pre-downloads them at build time to bake them into the image.

Decision 6: 3-part cache (chunks.pkl + bm25.pkl + vectors.npy) over single pickle

Alternative considered: Single large pickle: `pickle.dump(full_index_object)`

Reasoning: The full `RetrievalIndex` object contains the `fastembed TextEmbedding` model, which itself holds an `onnxruntime InferenceSession`. ONNX sessions cannot be pickled — they're C++ objects. A single-object pickle fails at build time. Splitting into three serializable parts (chunk dicts, BM25Okapi instance, numpy vectors) avoids this. The cache loads in ~10-15s vs 5-8 min for a full rebuild from the raw corpus.

Decision 7: JSON mode for LLM planner output

Alternative considered: Free-text response, parse with `string.split("\n")` or regex

Reasoning: The original planner split the LLM response on newlines to extract sub-queries. LLMs with temperature > 0 produce inconsistent formatting — numbered lists, dashes, commas, trailing punctuation. Any of these breaks string-based parsing silently (wrong sub-queries, not an error). JSON mode forces the model to return valid JSON. `json.loads()` then provides a hard error if parsing fails, with a fallback to the original query. Deterministic, not brittle.

Decision 8: Groq free tier with tenacity retry (wait_chain 30s)

Alternative considered: Immediate retry with exponential backoff starting at 2s

Reasoning: Groq's free tier rate limit resets on a 30-second window. Retrying after 2-10s hits the same rate limit state — the retries fail identically and exhaust all attempts. `wait_chain(wait_fixed(30), wait_exponential(30-60s))` waits the full window on first retry, ensuring the limit has reset. This is the correct strategy for a windowed rate limiter, not an exponential one.

05 Three Real Production Failures

These are not hypothetical improvements. Each failure actually occurred during development or deployment. The root cause was diagnosed, a fix was implemented and committed, and the lesson generalizes beyond this project.

Failure 1: Planner String-Parsing Fragility

What broke	Planner returned malformed sub-queries — sometimes just one mangled string instead of 3 clean sub-queries. The retrieval fan-out then ran on garbage input.
Root cause	The planner prompt asked for sub-queries and split the LLM response on '\n'. The model's output format varied: sometimes numbered ('1. ...'), sometimes dashed, sometimes comma-separated. Any format mismatch silently produced wrong splits — no error raised, just bad sub-queries.
Diagnosis	Inspected the raw LLM response text during a multi-query eval run. Saw output like '1. How do I... 2. What is...' being split into a single string because newlines were inconsistent across model calls.
Fix	Switched to JSON mode (response_format={type: json_object}) and json.loads() parsing. Added explicit fallback: if json.loads() fails or sub_queries is empty, fall back to original query. Failure is now explicit, not silent.
Lesson	Never parse LLM output with string heuristics when structured output is available. JSON mode turns a parsing failure into an exception — silence is the worst failure mode in LLM orchestration.

Failure 2: Groq Rate Limit Retry Too Fast

What broke	Under load, requests consistently failed with 429 Rate Limit from Groq. The retry logic exhausted all 3 attempts and raised the error to the caller, returning a 500 to the API client.
------------	---

Root cause	The tenacity retry was configured with <code>wait_exponential(multiplier=1, min=2, max=10)</code> . Groq's free tier rate limit resets on a 30-second window. Retrying at 2s, 4s, 8s means all retries land within the same rate limit window and fail identically. The exponential backoff was solving the wrong problem (server overload) when the actual constraint was a time-windowed rate limit.
Diagnosis	Checked Groq rate limit documentation: 30 requests/min on the free tier, enforced on a rolling 30s window. The retry strategy assumed server-side overload which resolves in seconds — incorrect for a windowed limit.
Fix	<code>wait_chain(wait_fixed(30), wait_exponential(multiplier=2, min=30, max=60))</code> . First retry waits the full 30s window. If still failing, exponential from 30s. This is the correct strategy for a windowed rate limiter.
Lesson	The correct retry strategy depends on the failure mode. Server overload → exponential backoff from small values. Windowed rate limit → wait ≥ window duration. Applying exponential backoff to a windowed limit is actively harmful — it uses all retry budget within the same failed window.

Failure 3: Container Cold Start Blocked by Index Rebuild

What broke	After deploying to Cloud Run, the <code>/health</code> endpoint returned <code>{cache: 'building'}</code> indefinitely. The service appeared healthy but all <code>/query</code> requests timed out. Logs showed: <code>'Fetching 5 files: 20% 1/5 [00:24]'</code> .
Root cause	Two compounding issues: (1) The startup event triggered <code>_get_index()</code> synchronously in the request handler thread, blocking the entire event loop while the index built. (2) The fastembed ONNX model download (5 files, ~25s each = ~2 minutes) was triggered at runtime even with the 3-part cache loaded, because <code>_load_from_cache()</code> calls <code>TextEmbedding(EMBEDDING_MODEL)</code> on line 72 to initialize the query-time embedding model — and the Dockerfile pre-download was writing to a different path than the runtime expected.
Diagnosis	Cloud Run logs via 'gcloud logging read'. Saw the ONNX download progress bar in stdout. Traced to <code>retriever_tool.py</code> line 72. Checked Dockerfile: pre-download runs <code>python3 -c</code> but doesn't set <code>FASTEMBED_CACHE_PATH</code> , so files land in a default temp path that differs from the runtime path.

Fix applied	Moved <code>_get_index()</code> call to a background daemon thread (<code>threading.Thread</code> with <code>daemon=True</code>) in the startup event. This unblocks the event loop. The 3-part cache (<code>chunks.pkl</code> , <code>bm25.pkl</code> , <code>vectors.npy</code>) is committed to git so the bulk of index loading (~10-15s) happens from cache, not rebuild.
Fix pending	<code>FASTEMBED_CACHE_PATH</code> env var alignment between Dockerfile build step and runtime. The model download should be eliminated entirely in cold start by ensuring the pre-downloaded files are found at runtime.
Lesson	In containerized ML services: (1) Never block the event loop with initialization — always use background threads. (2) Verify model cache paths are identical between build and runtime. <code>FASTEMBED_CACHE_PATH</code> must be explicitly set and consistent. (3) Health endpoints that check memory state (<code>is _index</code> not <code>None</code> ?) are more useful than file-existence checks — they tell you if the service is actually ready to serve queries.

06 Evaluation Results

200 WixQA queries were evaluated against the full pipeline. Results are stored in `outputs/evidence/` as JSON files (`eval_0000.json` through `eval_0199.json`). Each file captures the retrieval result, NLI decision, and metadata.

Summary Metrics

Eval set size	200 queries from WixQA dataset
ANSWER_WITH_CITATION	112 / 200 = 56% — NLI threshold met, citation returned
ABSTAIN	88 / 200 = 44% — NLI threshold not met, principled abstain
Citation precision	75.9% — fraction of returned citations that pass NLI verification
TOP_K_RETRIEVE	10 chunks retrieved per sub-query
TOP_K_VERIFY	3 chunks passed to NLI (top 3 by RRF score)
NLI threshold	confidence ≥ 0.85 AND verdict == SUPPORTS
NLI model	cross-encoder/nli-deberta-v3-small (HuggingFace, ~180MB)
Embedding model	BAAI/bge-small-en-v1.5 via fastembed ONNX (384-dim, ~130MB)
Corpus	6,221 WixQA help articles

Interpreting the ABSTAIN Rate

A 44% ABSTAIN rate is not a failure metric — it is a trust calibration metric. It means the system declines to answer 44% of the time rather than generating a citation-unsupported response. The alternative (always answer) would produce 56% verified answers and 44% unverified answers that look identical in the API response. ABSTAIN makes the failure mode explicit and recoverable.

The high ABSTAIN rate on WixQA has a structural cause: some queries ask about account ownership, billing disputes, or policy topics where the corpus articles describe process but don't directly 'entail' specific claims at the 0.85 confidence threshold. NLI at 0.85 is deliberately strict.

Interpreting Citation Precision

75.9% citation precision means that in 76 out of 100 cases where PulseAgent returns a citation, the cited chunk actually entails the answer (by the NLI model's judgment at 0.85 confidence). This is not

ground-truth precision — it is NLI-verified precision. The NLI model itself is not infallible (it fails on numeric hedging, low-confidence edge cases, and paraphrased claims). A human evaluation would be needed for ground-truth precision.

07 Known Limitations and Truth Boundary

This section is the honest accounting. Use these in interviews when asked 'what would you improve?' or 'what are the limitations?' — specificity here demonstrates real ownership. Vague answers suggest you don't know what's wrong.

What is real and verified

- ✓ Live deployed service at the Cloud Run URL — callable right now with the API key
- ✓ NLI citation verification runs on every request — not stubbed or mocked
- ✓ Parallel fan-out via Send() is wired and functional — planner does dispatch multiple sub-queries
- ✓ Auth (X-API-Key), rate limiting (10/min), structured logging — all in production code
- ✓ 3-part cache committed to git — cold start is ~10-15s from cache (not 5-8 min rebuild)
- ✓ 200-query evaluation results committed in outputs/evidence/ — the JSON files are real
- ✓ Self-reflective synthesis loop — generator and reflector both call the LLM

Known limitations

Single-domain corpus: WixQA only. The system has no transfer capability to other domains without re-indexing.

In-memory Qdrant: The Qdrant instance is :memory: — it rebuilds from the saved vectors.npy on every cold start. No persistence, no clustering. For production scale, Qdrant Cloud or a persistent deployment is needed.

fastembed cold-start: If the ONNX model cache misses at runtime (FASTEMBED_CACHE_PATH mismatch), the container takes 2+ minutes to become ready. This is a known open issue.

No RAGAS or formal eval: The 200-query eval measures NLI-verified precision, not answer quality. No faithfulness score, no answer relevance score, no BLEU/ROUGE.

MCP server untested at scale: The fastmcp MCP server is smoke-tested locally but not load-tested or deployed as a production endpoint.

Groq free tier: Subject to rate limits (~30 req/min shared). Under load, requests will hit 429. The retry handles it but adds latency. Not suitable for high-throughput production.

NLI on numeric claims: The NLI model (even at 0.85 threshold) is unreliable for numeric policy claims with hedging language ('up to X', 'at least Y'). The entailment.py file has a NumericPolicyVerifier for this, but it is not wired into the main pipeline yet.

What is not claimed

- ✗ This is not a general-purpose RAG framework — it is purpose-built for WixQA
- ✗ Citation precision (75.9%) is NLI-measured, not human-evaluated — do not cite it as ground truth
- ✗ The eval latency (avg 0.2s in eval files) is retrieval+NLI only — full pipeline with LLM takes 10-30s
- ✗ The project does not demonstrate fine-tuning, RLHF, or model training of any kind
- ✗ The ABSTAIN decision is not infallible — it is limited by the NLI model's accuracy

08 Hard Interview Q&A;

These are the questions an interviewer will actually ask. The answers below are the target — study them until you can say them without looking. Specificity wins. Vague answers ('I used RAG because it reduces hallucinations') signal that you read about the technique but didn't build it.

Q: Walk me through your architecture.

The system is a 4-stage LangGraph pipeline. A planner node decomposes the query into up to 3 sub-queries using an LLM in JSON mode. Those sub-queries fan out in parallel to separate RetrievalAgent instances via LangGraph's Send() API. Each retrieval run does hybrid BM25 plus dense retrieval fused with RRF at $k=60$. An aggregate node deduplicates by article_id, keeps highest RRF score per chunk. A verifier runs NLI with cross-encoder/nli-deberta-v3-small against the top 3 chunks. If any chunk passes at 0.85 confidence, it's ANSWER_WITH_CITATION. Otherwise ABSTAIN. A synthesis agent then generates and self-reflects on the answer.

Q: Why RRF instead of a weighted linear combination of BM25 and cosine scores?

BM25 scores and cosine similarities are on incomparable scales. Normalizing them requires a mixing weight hyperparameter alpha that would need tuning on a labeled dev set I don't have. RRF operates only on ranks — it's normalization-free and hyperparameter-free. $k=60$ is the standard from Cormack et al. 2009. Empirically, RRF performs well when the two retrieval systems are complementary, which BM25 and dense retrieval are — BM25 handles exact technical terms, dense handles semantic paraphrases.

Q: Why does ABSTAIN exist? Couldn't you just always generate an answer?

If I always generated an answer, users would get two types of responses: ones supported by the corpus and ones that aren't — and they'd look identical. ABSTAIN makes the failure mode explicit. When NLI says the retrieved chunks don't entail the query at 0.85 confidence, I don't have verified evidence to answer from. Generating anyway is fabrication with false confidence. ABSTAIN is better for trust calibration — users know when the system doesn't have an answer rather than receiving a plausible-sounding but unverified response.

Q: What's the NLI premise-hypothesis convention and why does it matter for questions?

In NLI, the premise is the evidence (the retrieved chunk) and the hypothesis is the claim to be verified. The model is trained on declarative claim pairs. A question like 'Can I add a payment method?' is interrogative — NLI models perform poorly on interrogative hypotheses because the training distribution is declarative claims. I convert the question to a declarative claim: 'This article provides information about: Can I add a payment method'. Strip the question mark, add the prefix. This aligns with how the model was trained and produces more reliable entailment scores.

Q: How does LangGraph's Send() API work? Why not just use a for loop?

Send() lets a node return a Command that dispatches to multiple target nodes simultaneously, each with its own input state. The key difference from a for loop: the dispatched nodes run in parallel within the same LangGraph execution step. A Python for loop would be sequential. The other critical piece is the Annotated reducer on all_retrieved_chunks — Annotated[List[dict], operator.add] tells LangGraph to concatenate all writes to that key, not overwrite. Without it, whichever parallel retrieval run finished last would overwrite the others.

Q: Your citation precision is 75.9%. Is that good? What does it mean?

75.9% means that in 76 out of 100 cases where the system returns a citation, the cited chunk actually entails the claim by the NLI model's judgment at 0.85 confidence. It's NLI-measured precision, not human-evaluated — so it's a lower bound on quality (the NLI model itself has errors, especially on numeric hedging). Whether 75.9% is good depends on the use case. For a help desk assistant where wrong citations erode trust quickly, it's not high enough — I'd want to raise the threshold or add a reranker. But it's better than no verification, where you'd return whatever the top-ranked chunk was regardless of relevance.

Q: Walk me through what happens when Groq returns a 429.

The LLM call is wrapped in a tenacity @retry decorator. Configuration: stop_after_attempt(3), wait_chain(wait_fixed(30), wait_exponential(30-60s)). On the first 429, tenacity waits 30 seconds — the full Groq rate limit window. If the second attempt also fails, it waits between 30 and 60 seconds with exponential growth. The original implementation used wait_exponential starting at 2 seconds, which meant all 3 retries landed within the same 30s rate limit window and all failed. The fix is to wait the full window first. before_sleep_log logs a warning with the exception before each retry.

Q: What's the difference between a bi-encoder and a cross-encoder? Which do you use and where?

A bi-encoder embeds the query and document independently, then computes similarity between the vectors. You can pre-compute all document vectors offline — retrieval at query time is just an ANN lookup. I use a bi-encoder (BAAI/bge-small-en-v1.5) for retrieval. A cross-encoder takes the concatenated (query, document) as a single input and scores the pair jointly. Much more accurate but $O(n)$ at query time — you can't pre-compute. I use a cross-encoder (nli-deberta-v3-small) only for NLI verification on the top 3 retrieved chunks. Standard pipeline: bi-encoder to retrieve candidates, cross-encoder to re-rank or verify the shortlist.

Q: Why did you choose Qdrant over FAISS or a simple numpy dot-product?

Qdrant supports pre-filtering — filter by doc_type, version, or corpus_tag BEFORE the cosine ranking, not after. With FAISS or numpy, you'd retrieve the top-K by cosine similarity and then filter, which wastes retrieved slots if many results don't pass the filter. Qdrant's filter pushdown means the returned results are already filtered. The current deployment is in-memory (:memory:), which means it rebuilds from vectors.npy on cold start. For production, Qdrant Cloud or a persistent deployment would eliminate this.

Q: What's the 3-part cache and why not just pickle the whole index?

The RetrievalIndex object contains a fastembed TextEmbedding model, which internally holds an onnxruntime InferenceSession — a C++ object that cannot be pickled. Trying to pickle.dump(index) fails at build time. The fix is to split into three serializable parts: chunk dicts (no model references), BM25Okapi object (pure Python, picklable), and the numpy vector array. These load in ~10-15 seconds instead of 5-8 minutes for a full rebuild. All three files are committed to git so the cache survives container restarts without a rebuild step.

Q: What would you build next if you had another week?

Three things in priority order. First, fix the FASTEMBED_CACHE_PATH alignment between Dockerfile and runtime — the 2-minute cold start on cache miss is a real prod reliability issue. Second, add a proper eval harness using RAGAS or a custom G-Eval — current eval is retrieval+NLI precision only, no answer quality measurement. Third, wire in the NumericPolicyVerifier from entailment.py — it already exists but isn't called in the main pipeline. NLI alone is unreliable for numeric policy claims with hedging language like 'up to X' or 'at least Y'. The verifier does deterministic number extraction and handles this correctly.

Q: What was the hardest technical problem you solved in this project?

The cold-start blocking issue, because it had two compounding root causes that looked like one. The symptoms: /health showed 'building' indefinitely, logs showed the fastembed ONNX download (5 files, ~25s each). I initially thought the 3-part cache wasn't working — but the cache loaded fine. The actual problem was that _load_from_cache() calls TextEmbedding(EMBEDDING_MODEL) to initialize the query-time embedding model, and that constructor triggers the ONNX download regardless of whether the vectors are cached. The Dockerfile pre-download wrote to a different path than the runtime looked in. Two independent bugs that produced identical symptoms. Diagnosing it required reading Cloud Run logs line by line and tracing the exact line in retriever_tool.py that triggered the download.

09 What I'd Build Next

Ordered by impact. The first two are reliability fixes (prod-grade work). The rest are capability expansions.

Fix fastembed cold-start (FASTEMBED_CACHE_PATH alignment)

Priority: Reliability — high

Set FASTEMBED_CACHE_PATH to a known path in Dockerfile, verify same path is used at runtime. Pre-download verification: after model download in Dockerfile, run a warmup inference call and assert it uses the cached path. Eliminates the 2-minute cold start on cache miss.

Proper evaluation harness (RAGAS or custom G-Eval)

Priority: Quality measurement — high

Current eval measures retrieval+NLI precision only. RAGAS provides: faithfulness (is the answer supported by context?), answer relevance (does the answer address the query?), context precision, and context recall. Without this, there's no way to measure whether changes to the pipeline improve or degrade answer quality.

Wire NumericPolicyVerifier into main pipeline

Priority: Correctness — medium

entailment.py has a complete NumericPolicyVerifier with deterministic number extraction, hedge detection ('up to X', 'at least Y'), and a CitationRoutingDecision layer. It's not connected to the verifier_agent. Wiring it would catch a class of NLI failures on policy claims with numeric thresholds.

Persistent Qdrant (Qdrant Cloud or self-hosted)

Priority: Architecture — medium

Current Qdrant is :memory: — rebuilds from vectors.npy on cold start. Persistent Qdrant means no rebuild on container restart, enables incremental corpus updates without a full re-index, and supports filtering at scale without the in-memory size constraint.

Multi-domain corpus support

Priority: Capability — low (for current scope)

PulseAgent is WixQA-only. Multi-domain would require: corpus_tag-based pre-filtering in retrieval, domain routing in the planner (which corpus to search), and separate eval sets per domain. The infrastructure already supports corpus_tag filtering in Qdrant — it's an architectural extension, not a rewrite.

Final note

This document covers what was built, why each decision was made, what broke and how it was fixed, what is known to be imperfect, and where the work goes next. Own all of it — the failures as much as the successes. An interviewer who asks about the cold-start issue is testing whether you understand your own system. The answer in Section 5 is the answer. Know it cold.